## 1. Conceitos Fundamentais

Um sistema multithread caracteriza-se pela existência de múltiplas threads de execução compartilhando o contexto de um mesmo processo. Enquanto o processo é a entidade que agrupa recursos (como espaço de endereçamento, arquivos abertos e dispositivos), a thread é a unidade básica de uso da CPU (escalonamento). As threads de um mesmo processo compartilham seu código, dados e outros recursos do sistema operacional. Assim como os processos, as threads possuem estados de execução principais: Em execução (usando a CPU), Pronta (aguardando a CPU) e Bloqueada (aguardando um evento de I/O ou sincronização).

## 2. Concorrência vs Paralelismo

É vital distinguir Concorrência de Paralelismo. Concorrência é a ilusão de execução simultânea em um único processador (multiprogramação através da troca rápida de contexto). Paralelismo exige hardware com múltiplos núcleos, onde as tarefas são de fato processadas no mesmo instante físico. Criar múltiplas threads pode facilitar a concorrência, mas não adianta criar um número massivo de threads esperando ganho de desempenho linear se o processador não suporta esse nível de paralelismo real.
Nesse contexto, avalia-se a escalabilidade em multiprocessadores, categorizando a divisão de trabalho em: Paralelismo de Dados (mesma operação aplicada em fatias diferentes de um grande conjunto de dados) e Paralelismo de Tarefas (diferentes threads executando rotinas distintas concorrentemente). A granularidade define o tamanho da tarefa atribuída a cada thread; ajustes inadequados podem gerar muito overhead de gerenciamento.


## 3. Modelos de Threads

O suporte às threads pode ser provido no nível do usuário ou pelo kernel. A relação entre essas threads define três modelos arquiteturais principais:
Muitos-para-Um (N:1): Várias threads de usuário mapeadas para uma única thread de kernel. O gerenciamento é eficiente, mas se uma thread bloquear, todo o processo bloqueia.
Um-para-Um (1:1): Cada thread de usuário mapeia para uma thread de kernel. Oferece alto paralelismo, mas a criação de muitas threads pode sobrecarregar o sistema.
Muitos-para-Muitos (N:M): Multiplexa muitas threads de usuário para um número igual ou menor de threads de kernel. Tenta reunir o melhor dos dois mundos.

## 4. Casos de Uso e Eficiência

A aplicação de multithreading não é universalmente vantajosa. Um "Caminho bom" é a multiplicação de matrizes, pois é um problema altamente paralelizável (paralelismo de dados), onde diferentes blocos podem ser processados independentemente sem contenção de recursos. Em contrapartida, um "Caminho ruim" é a busca em profundidade (DFS - Depth-First Search) em árvores não balanceadas, cuja natureza sequencial e dependente

torna o paralelismo complexo e ineficiente devido ao alto overhead de sincronização.

## 5. Implementação: Pthreads (POSIX)

A biblioteca Pthreads define o padrão para criação e sincronização de threads em sistemas UNIX. O comando padrão para compilação via GCC requer a vinculação da biblioteca:
gcc -Wall -pthread codigo.c -o codigo
Nas funções executadas por threads na API POSIX, utiliza-se amplamente o ponteiro void*. Ele é usado como argumento e retorno genérico de funções (ex: pthread_create), pois não se sabe previamente qual o tipo de dado específico que o desenvolvedor precisará passar (um inteiro, uma estrutura complexa ou um vetor). O void* permite passar qualquer endereço de memória com segurança.